

AeroSafe – Noise Management API

System Architecture, Domain Workflows, and Technical Interview Blueprint

1. System Overview & Technology Stack

AeroSafe is a modular airport operations backend inspired by industrial aerodrome compliance and monitoring systems. It manages core airport entities, monitors auditory emission thresholds, automatically dispatches compliance audits, and exposes a sandboxed LLM interface for equipment telemetry interrogation.

Layer	Technology Choice & Justification
Core Backend	Python, Django, Django REST Framework (DRF)
Database	SQLite (Dev) / PostgreSQL (Target Production)
Auth & Access	JWT (djangorestframework-simplejwt) + Django Groups (RBAC)
Documentation	drf-spectacular (OpenAPI 3.0 / Swagger / ReDoc)
Query Optimization	django-filter, select_related, prefetch_related, Composite Indexes
Local AI/LLM	Ollama orchestrating Llama 3.2 3B via decoupled Service Layer
Async & Cache	Celery + Redis (Architectural Roadmap)

2. Operational Business Workflow

The application models physical aerodrome operations via an event-driven entity chain:

- **Noise Sensor Telemetry:** Station records acoustic levels (e.g., 82.50 dB on Runway 12).
- **Threshold Breach:** NoiseLog record is persisted with OPEN state.
- **Inspection Trigger:** Operations staff logs a mandatory Noise Compliance Inspection against the incident.
- **Maintenance Action:** Inspection findings produce an actionable Work Order assigned to technical personnel.
- **Resolution:** Field engineering verifies sensor calibration or acoustic barriers and closes the loop.

3. Data Architecture & Integrity Decisions

Referential Integrity & Deletion Rules

- **Asset Deletion (SET_NULL):** on_delete=models.SET_NULL ensures historical acoustic telemetry logs survive device decommissions.
- **Audit Trail (PROTECT):** reported_by references the User model via on_delete=models.PROTECT, legally shielding historical compliance data from dangling author IDs.

Indexing Strategy

Composite index on ['airport', 'status'] and a single index on ['asset_type'] enable high-performance filtering across large IoT operational datasets without sequential table scans.

Server-Enforced User Attribution

Client payloads cannot specify `reported_by`. Identity is strictly derived server-side via `perform_create(self, serializer)` using `self.request.user` extracted from validated JWT claims.

```
def perform_create(self, serializer):
    serializer.save(reported_by=self.request.user)
```

4. Query Optimization & ORM Hygiene

To systematically prevent N+1 database queries across nested API views, QuerySets explicitly declare join semantics:

- **`select_related`**: Applied to direct foreign keys (`airport`, `asset`, `reported_by`) utilizing single SQL INNER/LEFT JOINS.
- **`prefetch_related`**: Applied to reverse foreign keys (`inspections`) to batch child collection queries in Python memory.

```
queryset = (
    NoiseLog.objects
    .select_related('airport', 'asset', 'reported_by')
    .prefetch_related('inspections')
)
```

5. Security, Authentication & RBAC

Stateless authentication is handled via short-lived JWT access tokens (30 min) and refresh tokens (24 h). Authorization separates authenticated state from permissions using custom DRF permission classes tied to Django Groups.

```
class IsOperationsUser(BasePermission):
    def has_permission(self, request, view):
        return bool(
            request.user and request.user.is_authenticated and
            (request.user.is_superuser or request.user.groups.filter(name='Operations').exists())
        )
```

6. AI / LLM Integration Pattern

Rather than coupling AI queries directly to ViewSets, the architecture enforces a strict Service Layer (`llm/services.py`). The service constructs context from the verified Asset model, injects guardrail system prompts ('Use only provided data; do not hallucinate'), and queries Ollama running Llama 3.2 3B. This abstraction allows swapping local Ollama for AWS Bedrock without altering the REST interface.

7. Architectural Interview Cheatsheet

Q: Why `select_related` vs. `prefetch_related`?

`select_related` performs SQL joins for single-valued relationships (ForeignKey, OneToOne). `prefetch_related` executes a separate batch query for multi-valued relationships (reverse FK, ManyToMany) and joins them in memory, avoiding N+1 bottlenecks.

Q: Why enforce server-side attribution for reported_by?

Accepting user IDs from client JSON violates trust boundaries and invites identity spoofing. We extract and assign request.user securely from the cryptographically verified JWT.

Q: Why separate LLM calls into a Service Layer?

To keep ViewSets focused solely on HTTP request/response validation. The service layer decouples the LLM provider, making unit testing simpler and enabling seamless migration to enterprise cloud providers like AWS Bedrock.